LAPORAN PRATIKUM ALGORITMA PEMROGRAMAN

ADT

Disusun Oleh:

Endy Pardilian

2511531017Dosen Pengampu:

Wahyudi. Dr., S.T,M.T

Asisten Pratikum:

M Galid Avero

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

KATA PENGANTAR

Puji syukur penulis ucapkan kepada Tuhan Yang Maha Esa karena atas rahmat-Nya laporan praktikum ini dapat diselesaikan dengan baik. Laporan ini dibuat sebagai bentuk dokumentasi dan pertanggungjawaban atas kegiatan praktikum algoritma pemrograman yang telah dilakukan.

Pada praktikum ini, penulis mempelajari tentang implementasi Abstract Data Type (ADT) menggunakan bahasa pemrograman Java, khususnya pada objek Jam dan Mobil. Melalui praktikum ini, penulis dapat memahami bagaimana membuat kelas, atribut, method, serta mengelola data menggunakan konsep objek.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran sangat diharapkan untuk perbaikan ke depannya.

Padang, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR 2

DAFTAR ISI 3

BAB I PENDAHULUAN 1

1.1 Latar Belakang 1

1.2 Tujuan 1

1.3 Manfaat 1

BAB II PEMBAHASAN 2

2.1 Jam_2511531017 2

BAB III KESIMPULAN 14

3.1 Kesimpulan 15

3.2 Saran 15

DAFTAR PUSTAKA 15

BAB IPENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman berorientasi objek, Abstract Data Type (ADT) merupakan konsep penting yang digunakan untuk merepresentasikan suatu objek beserta operasi-operasi yang dapat dilakukan terhadap objek tersebut.

Pada praktikum ini, dilakukan implementasi ADT menggunakan bahasa Java dengan membuat dua jenis objek, yaitu Jam dan Mobil. ADT Jam digunakan untuk merepresentasikan waktu dalam satuan jam, menit, dan detik, serta menyediakan berbagai operasi seperti penjumlahan waktu, pengurangan, dan konversi ke detik.

Sedangkan ADT Mobil digunakan untuk mengelola data kendaraan, seperti nama, tahun, kapasitas mesin (cc), harga, dan merk, serta mengelola kumpulan data mobil menggunakan struktur ArrayList.

1.2 Tujuan

1. Memahami konsep Abstract Data Type (ADT)
2. Mengimplementasikan ADT dalam bahasa Java
3. Membuat program manipulasi objek Jam
4. Membuat program pengelolaan data Mobil menggunakan ArrayList

1.3 Manfaat

1. Melatih kemampuan pemrograman berorientasi objek
2. Memahami penggunaan class, object, dan method
3. Meningkatkan kemampuan dalam mengelola data dalam program
4. Menjadi dasar untuk pembuatan program yang lebih kompleks

BAB II PEMBAHASAN

2.1 Jam_2511531017

```
package pekan1_2511531017;

public class Jam_2511531017 {
	private int hh;
	private int mm;
	private int ss;

	public static boolean isValid(int h, int m, int s) {
		return (0 <= h && h <= 23) && (0 <= m && m <= 59) && (0 <= s &&
s <= 59);
	}

	public Jam_2511531017(int h, int m, int s) {
		this.hh= h; this.mm= m; this.ss= s;
	}

	public int getHour() {return hh;}
	public int getMinute() {return mm;}
	public int getSecond() {return ss;}

	public void SetHour(int h) {this.hh =h;}
	public void SetMinute(int m) {this.mm =m;}
	public void SetSecond(int s) {this.ss =s;}

	public int toSeconds() {return hh * 3600 + mm * 60 + ss;}
	public static Jam_2511531017 fromSeconds (int total){
		if (total < 0) throw new IllegalArgumentException("detik
negatif");
		total %= 24 * 3600;
		int h = total /3600; total %= 3600;
		int m = total /60;
		int s = total % 60;
		return new Jam_2511531017(h,m,s);
	}

	public int compareTo (Jam_2511531017 other) {return
Integer.compare(this.toSeconds(), other.toSeconds());}
	public boolean equals (Object o) {
		if (!(o instanceof Jam_2511531017 j)) return false;
		return hh == j.hh && mm == j.mm && ss == j.ss;
	}
	public int hashCode() {return java.util.Objects.hash(hh,mm,ss);}

	public Jam_2511531017 plus (Jam_2511531017 other) {return fromSeconds
(this.toSeconds() + other.toSeconds());}
	public Jam_2511531017 minus (Jam_2511531017 other) {return fromSeconds
(Math.floorMod(this.toSeconds() - other.toSeconds(), 24*3600));}
	public Jam_2511531017 nextSecond () {return fromSeconds(this.toSeconds()
+1); }
```

```
	public Jam_2511531017 nextNSecond (int n) {return fromSeconds
(this.toSeconds() + Math.max(0, n));}
	public Jam_2511531017 prevSecond () {return fromSeconds
(Math.floorMod(this.toSeconds() - 1, 24*3600));}
	public Jam_2511531017 prevNSecond (int n) {return fromSeconds
(Math.floorMod(this.toSeconds() - Math.max(0,n), 24*3600));}

	public static int durasiDetik(Jam_2511531017 jaw, Jam_2511531017 jakh)
{return jakh.toSeconds() - jaw.toSeconds();}

	public String toString() {return String.format("%02d:%02d:%02d", hh,
mm,ss);}

}
```

Code 2.1

Penjelasan:

```
	private int hh;
	private int mm;
	private int ss;

	public Jam_2511531017(int h, int m, int s) {
		this.hh= h; this.mm= m; this.ss= s;
	}
```

Kode di atas merupakan deklarasi atribut yang digunakan untuk menyimpan nilai jam, menit, dan detik. Ketiga atribut tersebut bertipe integer. Konstruktor digunakan untuk menginisialisasi nilai awal objek Jam ketika objek dibuat.

```
	public static boolean isValid(int h, int m, int s) {
		return (0 <= h && h <= 23) && (0 <= m && m <= 59) && (0 <= s &&
s <= 59);
	}
```

Method ini digunakan untuk memvalidasi apakah nilai jam, menit, dan detik yang dimasukkan sesuai dengan batasan waktu yang benar. Jika semua nilai berada dalam rentang yang valid, maka method akan mengembalikan nilai true, jika tidak maka false.

```
	public int getHour() {return hh;}
	public int getMinute() {return mm;}
	public int getSecond() {return ss;}

	public void SetHour(int h) {this.hh =h;}
	public void SetMinute(int m) {this.mm =m;}
	public void SetSecond(int s) {this.ss =s;}
```

Method getter digunakan untuk mengambil nilai dari atribut jam, menit, dan detik. Sedangkan method setter digunakan untuk mengubah nilai atribut tersebut. Hal ini merupakan penerapan konsep enkapsulasi dalam pemrograman berorientasi objek.

```
	public int toSeconds() {return hh * 3600 + mm * 60 + ss;}
	public static Jam_2511531017 fromSeconds (int total){
		if (total < 0) throw new IllegalArgumentException("detik
negatif");
		total %= 24 * 3600;
		int h = total /3600; total %= 3600;
		int m = total /60;
		int s = total % 60;
		return new Jam_2511531017(h,m,s);
	}
```

Method toSeconds digunakan untuk mengubah waktu dalam bentuk jam, menit, dan detik menjadi total detik. Sedangkan method fromSeconds digunakan untuk mengubah nilai detik kembali menjadi format jam. Method ini juga memastikan bahwa waktu tetap berada dalam rentang 24 jam.

```
	public int compareTo (Jam_2511531017 other) {return
Integer.compare(this.toSeconds(), other.toSeconds());}
	public boolean equals (Object o) {
		if (!(o instanceof Jam_2511531017 j)) return false;
		return hh == j.hh && mm == j.mm && ss == j.ss;
	}
```

Method compareTo digunakan untuk membandingkan dua objek Jam berdasarkan jumlah detiknya. Sedangkan method equals digunakan untuk mengecek apakah dua objek Jam memiliki nilai jam, menit, dan detik yang sama.

```
	public Jam_2511531017 plus (Jam_2511531017 other) {return fromSeconds
(this.toSeconds() + other.toSeconds());}
	public Jam_2511531017 minus (Jam_2511531017 other) {return fromSeconds
(Math.floorMod(this.toSeconds() - other.toSeconds(), 24*3600));}
```

Method plus digunakan untuk menjumlahkan dua waktu, sedangkan method minus digunakan untuk mengurangkan waktu. Hasil dari operasi tersebut akan dikonversi kembali ke format jam menggunakan method fromSeconds.

```
	public Jam_2511531017 nextSecond () {return fromSeconds(this.toSeconds()
+1); }
	public Jam_2511531017 nextNSecond (int n) {return fromSeconds
(this.toSeconds() + Math.max(0, n));}
	public Jam_2511531017 prevSecond () {return fromSeconds
(Math.floorMod(this.toSeconds() - 1, 24*3600));}
	public Jam_2511531017 prevNSecond (int n) {return fromSeconds
(Math.floorMod(this.toSeconds() - Math.max(0,n), 24*3600));}
```

Method-method di atas digunakan untuk menambah atau mengurangi waktu dalam satuan detik. nextSecond menambah satu detik, sedangkan prevSecond mengurangi satu detik. Untuk nextNSecond dan prevNSecond digunakan untuk menambah atau mengurangi beberapa detik sekaligus.

```
	public static int durasiDetik(Jam_2511531017 jaw, Jam_2511531017 jakh)
{return jakh.toSeconds() - jaw.toSeconds();}

	public String toString() {return String.format("%02d:%02d:%02d", hh,
mm,ss);}

}
```

Method durasiDetik digunakan untuk menghitung selisih waktu dalam satuan detik antara dua objek Jam. Sedangkan method toString digunakan untuk menampilkan waktu dalam format jam:menit:detik dengan dua digit.

2.2 JamDriver_2511531017

```
package pekan1_2511531017;

public class JamDriver_2511531017 {

	public static void main(String[] args) {
		// TODO Auto-generated method stub
		Jam_2511531017 a = new Jam_2511531017(23,59,50);
		Jam_2511531017 b = new Jam_2511531017(0,0,15);
		System.out.println("a		="+a);
		System.out.println("b		="+b);
		System.out.println("a+b		="+a.plus(b));
		System.out.println("next 20s	="+a.nextNSecond(20));
		System.out.println("durasi(a,b)
="+Jam_2511531017.durasiDetik(a, b));
		System.out.println("a.compareTo(b) ="+a.compareTo(b));
	}


	}
```

Code 2.2

Penjelasan:

```
		Jam_2511531017 a = new Jam_2511531017(23,59,50);
		Jam_2511531017 b = new Jam_2511531017(0,0,15);
		System.out.println("a		="+a);
		System.out.println("b		="+b);
```

Kode di atas digunakan untuk membuat dua objek Jam, yaitu objek a dan b dengan nilai waktu yang berbeda. Kemudian, kedua objek tersebut ditampilkan menggunakan method toString() secara otomatis sehingga menghasilkan output dalam format jam:menit:detik.

```
		System.out.println("a+b		="+a.plus(b));
		System.out.println("next 20s	="+a.nextNSecond(20));
```

Kode ini digunakan untuk menguji operasi penjumlahan dua objek Jam menggunakan method plus(). Selain itu, method nextNSecond() digunakan untuk menambahkan sejumlah detik tertentu ke objek a.

```
            System.out.println("durasi(a,b)
="+Jam_2511531017.durasiDetik(a, b));
            System.out.println("a.compareTo(b) ="+a.compareTo(b));
      }
```

Kode ini digunakan untuk menghitung selisih waktu antara objek a dan b dalam satuan detik menggunakan method durasiDetik(). Selain itu, method compareTo() digunakan untuk membandingkan dua objek waktu.

Output:

```
Console
<terminated> JamDriver_2511531017 [Java Application] C:\Users\SISI\.p
a               =23:59:50
b               =00:00:15
a+b             =00:00:05
next 20s        =00:00:10
durasi(a,b)     =-86375
a.compareTo(b) =1
```

Gambar 2.1

2.3 JamDriver2_2511531017

```
package pekan1_2511531017;

import java.util.Scanner;
public class JamDriver2_2511531017 {

      public static void main(String[] args) {
            // TODO Auto-generated method stub
            Scanner input = new Scanner(System.in);
            System.out.println("=== Program Driver Objek Jam ===");

            System.out.println("\n--- Input Jam 1 ---");
            Jam_2511531017 j1 = buatJamDariInput(input);

            System.out.println("\n--- Input Jam 2 ---");
            Jam_2511531017 j2 = buatJamDariInput(input);

            System.out.println("\n--- Hasil Operasi ---");
            System.out.println("Jam 1 (String)          :" + j1.toString());
            System.out.println("Jam 2 (String)          :" + j2.toString());
            System.out.println("Jam 1 dalam detik       :" + j1.toSeconds());
            System.out.println("Jam 2 dalam detik       :" + j2.toSeconds());

            int perbandingan = j1.compareTo(j2);
            if (perbandingan > 0) {
                  System.out.println("Status                : Jam 1 lebih
lambat (setelah) Jam 2");
            } else if (perbandingan < 0) {
                  System.out.println("Status                : Jam 1 lebih
awal (sebelum) Jam 2");
            } else {
```

```
                    System.out.println("Status                : Jam 1 dan Jam 2
saam persis");
            }
            System.out.println("Durasi (J1 ke J2)        :" +
Jam_2511531017.durasiDetik(j1,j2) + " detik");

            Jam_2511531017 jNext = j1.nextSecond();
            System.out.println("Jam 1 Detik Berikutnya   :" + jNext);

            Jam_2511531017 jPrev =j1.prevSecond();
            System.out.println("Jam 1 Detik Sebelumnya   :" + jPrev);

            Jam_2511531017 jHasilPlus = j1.plus(j2);
            System.out.println("Hasil J1 + J2            :" + jHasilPlus);

            input.close();
            System.out.println("\n Program Selesai.");
      }


      private static Jam_2511531017 buatJamDariInput (Scanner sc) {
            int h,m,s;
            while (true) {
                    System.out.println("Masukan Jam (0-23)  :");
                    h= sc.nextInt();

                    System.out.println("Masukan Menit (0-59)  :");
                    m= sc.nextInt();

                    System.out.println("Masukan Detik (0-59)  :");
                    s= sc.nextInt();

                    if (Jam_2511531017.isValid(h,m,s)) {
                            return new Jam_2511531017(h,m,s);
                    }else {
                            System.out.println("[Error] Input tidak valid!
Silahkan ulangi.\n");
                    }
      }

      }
}
```

Code 2.3

Penjelasan:

```
            Scanner input = new Scanner(System.in);
            System.out.println("=== Program Driver Objek Jam ===");

            System.out.println("\n--- Input Jam 1 ---");
            Jam_2511531017 j1 = buatJamDariInput(input);

            System.out.println("\n--- Input Jam 2 ---");
            Jam_2511531017 j2 = buatJamDariInput(input);
```

Kode di atas digunakan untuk menerima input dari pengguna menggunakan objek Scanner. User diminta memasukkan dua data waktu yang kemudian akan disimpan dalam objek j1 dan j2.

```
System.out.println("\n--- Hasil Operasi ---");
System.out.println("Jam 1 (String)          :" + j1.toString());
System.out.println("Jam 2 (String)          :" + j2.toString());
System.out.println("Jam 1 dalam detik       :" + j1.toSeconds());
System.out.println("Jam 2 dalam detik       :" + j2.toSeconds());
```

Kode ini digunakan untuk menampilkan data waktu dalam bentuk string serta dalam bentuk total detik. Hal ini menunjukkan fungsi konversi dari objek Jam.

```
int perbandingan = j1.compareTo(j2);
if (perbandingan > 0) {
        System.out.println("Status                : Jam 1 lebih
lambat (setelah) Jam 2");
} else if (perbandingan < 0) {
        System.out.println("Status                : Jam 1 lebih
awal (sebelum) Jam 2");
} else {
        System.out.println("Status                : Jam 1 dan Jam 2
saam persis");
}
```

Kode ini digunakan untuk membandingkan dua objek Jam. Hasil perbandingan akan menentukan apakah Jam 1 lebih cepat, lebih lambat, atau sama dengan Jam 2.

```
System.out.println("Durasi (J1 ke J2)        :" +
Jam_2511531017.durasiDetik(j1,j2) + " detik");

Jam_2511531017 jNext = j1.nextSecond();
System.out.println("Jam 1 Detik Berikutnya   :" + jNext);

Jam_2511531017 jPrev =j1.prevSecond();
System.out.println("Jam 1 Detik Sebelumnya   :" + jPrev);

Jam_2511531017 jHasilPlus = j1.plus(j2);
System.out.println("Hasil J1 + J2            :" + jHasilPlus);

input.close();
System.out.println("\n Program Selesai.");
}
```

Kode di atas digunakan untuk menguji berbagai operasi pada objek Jam, seperti menghitung durasi, menambah dan mengurangi detik, serta menjumlahkan dua waktu.

```
private static Jam_2511531017 buatJamDariInput (Scanner sc) {
        int h,m,s;
        while (true) {
                System.out.println("Masukan Jam (0-23)  :");
                h= sc.nextInt();
```

```
                    System.out.println("Masukan Menit (0-59)  :");
                    m= sc.nextInt();

                    System.out.println("Masukan Detik (0-59)  :");
                    s= sc.nextInt();

                    if (Jam_2511531017.isValid(h,m,s)) {
                          return new Jam_2511531017(h,m,s);
                    }else {
                          System.out.println("[Error] Input tidak valid!
Silahkan ulangi.\n");
                    }
      }
```

Method ini digunakan untuk menerima input waktu dari pengguna dengan validasi. Jika input tidak sesuai, maka pengguna diminta untuk mengulangi hingga memasukkan data yang benar.

Output:

```
Console
<terminated> JamDriver2_2511531017 [Java Application] C:\Users\SISI\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.9.v20251105-0741\jre\bin\ja
=== Program Driver Objek Jam ===

--- Input Jam 1 ---
Masukan Jam (0-23)  :
4
Masukan Menit (0-59)  :
17
Masukan Detik (0-59)  :
4

--- Input Jam 2 ---
Masukan Jam (0-23)  :
17
Masukan Menit (0-59)  :
4
Masukan Detik (0-59)  :
17

--- Hasil Operasi ---
Jam 1 (String)          :04:17:04
Jam 2 (String)          :17:04:17
Jam 1 dalam detik       :15424
Jam 2 dalam detik       :61457
Status               : Jam 1 lebih awal (sebelum) Jam 2
Durasi (J1 ke J2)        :46033 detik
Jam 1 Detik Berikutnya   :04:17:05
Jam 1 Detik Sebelumnya   :04:17:03
Hasil J1 + J2            :21:21:21

 Program Selesai.
```

Gambar 2.2

2.4 Mobil_2511531017

```
package pekan1_2511531017;
import java.util.*;

public class Mobil_2511531017 {

    private String nama;
    private int tahun;
    private int cc;
    private long harga;
    private String merk;

    private ArrayList<Mobil_2511531017> listMobil;

    public Mobil_2511531017() {
```

```
        listMobil = new ArrayList<>();
    }

    public Mobil_2511531017(String nama, int tahun, int cc, long harga,
String merk) {
        this.nama = nama;
        this.tahun = tahun;
        this.cc = cc;
        this.harga = harga;
        this.merk = merk;
    }

    public String getNama() {
       return nama;
       }
    public int getTahun() {
       return tahun;
       }
    public int getCc() {
       return cc;
       }
    public long getHarga() {
       return harga;
       }
    public String getMerk() {
       return merk;
       }

    public void setNama(String nama) {
       this.nama = nama;
       }
    public void setTahun(int tahun) {
       this.tahun = tahun;
       }
    public void setCc(int cc) {
       this.cc = cc;
       }
    public void setHarga(long harga) {
       this.harga = harga;
       }
    public void setMerk(String merk) {
       this.merk = merk;
       }

    public void tambah(Mobil_2511531017 m) {
        listMobil.add(m);
        System.out.println("Ditambah: " + m.getNama());
    }

    public void hapus(Mobil_2511531017 m) {
        if (listMobil.remove(m)) {
            System.out.println("Dihapus: " + m.getNama());
        } else {
            System.out.println("Data ga ketemu");
        }
    }
```

```
    public void tampil() {
        System.out.println("\nData mobil:");
        for (Mobil_2511531017 m : listMobil) {
            System.out.println(m);
        }
    }
    @Override
    public String toString() {
        return nama + " | " + tahun + " | " + cc + "cc | Rp" + harga + " | "
+ merk;
    }
}
```

Code 2.4

Penjelasan:

```
    private String nama;
    private int tahun;
    private int cc;
    private long harga;
    private String merk;

    private ArrayList<Mobil_2511531017> listMobil;

    public Mobil_2511531017() {
        listMobil = new ArrayList<>();
    }

    public Mobil_2511531017(String nama, int tahun, int cc, long harga,
String merk) {
        this.nama = nama;
        this.tahun = tahun;
        this.cc = cc;
        this.harga = harga;
        this.merk = merk;
    }
```

Kode di atas merupakan deklarasi atribut yang menyimpan informasi tentang mobil, seperti nama, tahun, kapasitas mesin, harga, dan merk. Selain itu, terdapat ArrayList yang digunakan untuk menyimpan kumpulan objek mobil.

```
    public String getNama() {
       return nama;
       }
    public int getTahun() {
       return tahun;
       }
    public int getCc() {
       return cc;
       }
    public long getHarga() {
       return harga;
       }
    public String getMerk() {
       return merk;
       }
```

```
public void setNama(String nama) {
   this.nama = nama;
   }
public void setTahun(int tahun) {
   this.tahun = tahun;
   }
public void setCc(int cc) {
   this.cc = cc;
   }
public void setHarga(long harga) {
   this.harga = harga;
   }
public void setMerk(String merk) {
   this.merk = merk;
   }
```

Method getter digunakan untuk mengambil data dari atribut mobil, sedangkan setter digunakan untuk mengubah nilai atribut tersebut. Hal ini memudahkan pengelolaan data dalam objek.

```
public void tambah(Mobil_2511531017 m) {
    listMobil.add(m);
    System.out.println("Ditambah: " + m.getNama());
}
```

Method ini digunakan untuk menambahkan objek mobil ke dalam ArrayList. Setiap data yang ditambahkan akan langsung dimasukkan ke dalam list dan ditampilkan sebagai konfirmasi.

```
public void hapus(Mobil_2511531017 m) {
    if (listMobil.remove(m)) {
        System.out.println("Dihapus: " + m.getNama());
    } else {
        System.out.println("Data ga ketemu");
    }
}
```

Method ini digunakan untuk menghapus data mobil dari ArrayList. Jika data ditemukan, maka akan dihapus dan ditampilkan pesan berhasil. Jika tidak ditemukan, maka akan muncul pesan bahwa data tidak ditemukan.

```
public void tampil() {
    System.out.println("\nData mobil:");
    for (Mobil_2511531017 m : listMobil) {
        System.out.println(m);
    }
}
```

Method tampil digunakan untuk menampilkan seluruh data mobil yang tersimpan di dalam ArrayList. Data akan ditampilkan satu per satu menggunakan perulangan.

```
@Override
public String toString() {
    return nama + " | " + tahun + " | " + cc + "cc | Rp" + harga + " | "
+ merk;
}
```

```
}
```

Method toString digunakan untuk mengatur format tampilan data mobil agar lebih rapi dan mudah dibaca ketika ditampilkan.

2.5 Driver_2511531017

```
package pekan1_2511531017;

public class Driver_2511531017 {

    public static void main(String[] args) {

        Mobil_2511531017 data = new Mobil_2511531017();

        Mobil_2511531017 m1 = new Mobil_2511531017("Raize", 2021, 1000,
200000000, "Toyota");
        Mobil_2511531017 m2 = new Mobil_2511531017("Brio", 2022, 1200,
170000000, "Honda");
        Mobil_2511531017 m3 = new Mobil_2511531017("Terios", 2020, 1500,
240000000, "Daihatsu");

        data.tambah(m1);
        data.tambah(m2);
        data.tambah(m3);

        data.tampil();

        data.hapus(m2);

        data.tampil();
    }
}
```

Code 2.5

Pembahasan:

```
        Mobil_2511531017 data = new Mobil_2511531017();

        Mobil_2511531017 m1 = new Mobil_2511531017("Raize", 2021, 1000,
200000000, "Toyota");
        Mobil_2511531017 m2 = new Mobil_2511531017("Brio", 2022, 1200,
170000000, "Honda");
        Mobil_2511531017 m3 = new Mobil_2511531017("Terios", 2020, 1500,
240000000, "Daihatsu");
```

Kode di atas digunakan untuk membuat objek utama sebagai tempat penyimpanan data mobil, serta membuat beberapa objek mobil dengan data yang berbeda.

```
        data.tambah(m1);
        data.tambah(m2);
        data.tambah(m3);
```

Kode ini digunakan untuk menambahkan data mobil ke dalam ArrayList melalui method tambah().

```
        data.tampil();
```

Kode ini digunakan untuk menampilkan seluruh data mobil yang telah ditambahkan ke dalam list.

```
        data.hapus(m2);
```

Kode ini digunakan untuk menghapus salah satu data mobil dari list, yaitu mobil m2.

```
    data.tampil();
 }
}
```

Kode ini digunakan untuk menampilkan kembali data mobil setelah dilakukan proses penghapusan, sehingga dapat terlihat perubahan data.

Output:

```
Console ×
<terminated> Driver_2511531017 [Java Application] C:\Users\SISI\.p2\pool\plugins\org.eclipse.justj
Ditambah: Raize
Ditambah: Brio
Ditambah: Terios

Data mobil:
Raize | 2021 | 1000cc | Rp200000000 | Toyota
Brio | 2022 | 1200cc | Rp170000000 | Honda
Terios | 2020 | 1500cc | Rp240000000 | Daihatsu
Dihapus: Brio

Data mobil:
Raize | 2021 | 1000cc | Rp200000000 | Toyota
Terios | 2020 | 1500cc | Rp240000000 | Daihatsu
```

Gambar 2.3

BAB IIIKESIMPULAN

3.1 Kesimpulan

Pada praktikum ini, dapat disimpulkan bahwa konsep Abstract Data Type (ADT) dapat diimplementasikan dengan baik menggunakan bahasa Java melalui pembuatan class dan object.

Program ADT Jam berhasil dibuat dengan berbagai operasi seperti konversi waktu, penjumlahan, pengurangan, dan perbandingan waktu. Sedangkan program ADT Mobil berhasil mengelola data kendaraan menggunakan struktur ArrayList dengan fitur tambah, hapus, dan tampil data.

Praktikum ini membantu dalam memahami konsep dasar pemrograman berorientasi objek serta pengelolaan data dalam program.

3.2 Saran

Sebagai saran, sebaiknya penjelasan materi saat praktikum bisa lebih detail dan perlahan, supaya mahasiswa yang belum terlalu paham coding bisa mengikuti dengan baik.

DAFTAR PUSTAKA

[1] Oracle, “Class Object (Java Platform SE),” [Online]. Tersedia: https://docs.oracle.com/javase/8/docs/api/java/lang/Object.html . [2025].

[2] Oracle, “ArrayList Class in Java,” [Online]. Tersedia: https://docs.oracle.com/javase/8/docs/api/java/util/ArrayList.html . [2025].